

**Moscow Aviation Institute
(National Research University)**

Faculty of Information Technologies and Applied Mathematics

Department of Computational Mathematics and Programming

Laboratory work No. 1, course “Numerical Methods”

Numerical Methods of Linear Algebra

Student: M. A. Khorokhorin

Teacher:

Group: M8O-306BV-24

Variant: 28

Date: 30.09.2026

Grade:

Signature:

Moscow, 2026

1 LU decomposition

1.1 Task

Implement the LU decomposition of a matrix with the choice of the pivot element. Use it to solve the system of linear algebraic equations, to compute the determinant and the inverse matrix.

$$\begin{cases} x_1 + 4x_2 - 9x_3 + 7x_4 = -67 \\ 2x_1 - 2x_2 - 2x_3 + 3x_4 = -57 \\ -x_1 + 3x_2 - 9x_3 - x_4 = -26 \\ -5x_1 + 2x_2 + 2x_3 + x_4 = 52 \end{cases}$$

1.2 Method

The matrix is factored as $PA = LU$, where L is a unit lower triangular matrix, U is an upper triangular matrix and P is a permutation matrix. At step $k = 1, \dots, n$ the row with the largest $|u_{ik}|$, $i \geq k$, is swapped with row k (partial pivoting), and then

$$l_{ik} = \frac{u_{ik}}{u_{kk}}, \quad u_{ij} \leftarrow u_{ij} - l_{ik}u_{kj}, \quad i, j > k.$$

Partial pivoting keeps $|l_{ik}| \leq 1$ and allows zeros on the diagonal of A . A pivot $|u_{kk}| \leq 10^{-12}\|A\|_\infty$ means that the matrix is singular to working precision: a relative perturbation of A comparable to rounding errors can make it exactly singular.

The system $Ax = b$ is solved by two triangular substitutions, $Lz = Pb$ and $Ux = z$:

$$z_i = (Pb)_i - \sum_{j < i} l_{ij}z_j, \quad x_i = \frac{1}{u_{ii}} \left(z_i - \sum_{j > i} u_{ij}x_j \right).$$

The determinant is $\det A = (-1)^s \prod_{i=1}^n u_{ii}$, where s is the number of row swaps. The j -th column of A^{-1} is the solution of $Ax = e_j$; all n systems reuse the same factorization, so the decomposition costs $O(n^3)$ once and every solution costs $O(n^2)$.

1.3 Results

The program reads n , the matrix A and the vector b and prints P as the order of rows of A , the factors L and U , the solution, the determinant and the inverse matrix.

Listing 1: tests/data/lu/01.in

```
4
1 4 -9 7
2 -2 -2 3
-1 3 -9 -1
-5 2 2 1
-67 -57 -26 52
```

Listing 2: tests/data/lu/01.out

```
P = (4, 1, 3, 2)
L =
  1.0000  0.0000  0.0000  0.0000
 -0.2000  1.0000  0.0000  0.0000
  0.2000  0.5909  1.0000  0.0000
 -0.4000 -0.2727  0.8211  1.0000
U =
 -5.0000  2.0000  2.0000  1.0000
  0.0000  4.4000 -8.6000  7.2000
  0.0000  0.0000 -4.3182 -5.4545
  0.0000  0.0000  0.0000  9.8421
x = (-5.0000, 9.0000, 7.0000, -5.0000)
det(A) = 935.0000
A^-1 =
  0.1027 -0.1979 -0.1112 -0.2364
  0.1786 -0.4171 -0.1102 -0.1091
  0.0396 -0.1283 -0.1262 -0.0182
  0.0770  0.1016 -0.0834  0.0727
```

The solution is $x = (-5, 9, 7, -5)^T$ and $\det A = 935$. The unit tests check $LU = PA$, $|l_{ik}| \leq 1$ and $AA^{-1} = E$ on random matrices up to 8×8 .

1.4 Source code

Listing 3: include/lu.hpp

```
1 | #pragma once
2 |
3 | #include <cstdint>
4 | #include <vector>
5 |
6 | #include "matrix.hpp"
7 |
```

```

8 | struct LuDecomposition {
9 |     Matrix lower;
10 |    Matrix upper;
11 |    std::vector<std::size_t> permutation;
12 |    int sign = 1;
13 | };
14 |
15 | LuDecomposition decomposeLu(const Matrix& matrix);
16 | Vector solveLu(const LuDecomposition& lu, const Vector& rhs);
17 | double determinant(const LuDecomposition& lu);
18 | Matrix inverse(const LuDecomposition& lu);

```

Listing 4: src/lu.cpp

```

1 | #include "lu.hpp"
2 |
3 | #include <cmath>
4 | #include <numeric>
5 | #include <stdexcept>
6 | #include <utility>
7 |
8 | namespace {
9 |
10 | constexpr double SINGULARITY_TOLERANCE = 1e-12;
11 |
12 | }
13 |
14 | LuDecomposition decomposeLu(const Matrix& matrix) {
15 |     if (!matrix.isSquare() || matrix.rows() == 0) {
16 |         throw std::invalid_argument("LU decomposition needs a non-empty square matrix");
17 |     }
18 |     const std::size_t n = matrix.rows();
19 |     const double threshold = SINGULARITY_TOLERANCE * normInf(matrix);
20 |     LuDecomposition result{.lower = Matrix::identity(n),
21 |                          .upper = matrix,
22 |                          .permutation = std::vector<std::size_t>(n),
23 |                          .sign = 1};
24 |     std::iota(result.permutation.begin(), result.permutation.end(), std::size_t{0});
25 |     Matrix& lower = result.lower;
26 |     Matrix& upper = result.upper;
27 |
28 |     for (std::size_t k = 0; k < n; ++k) {
29 |         std::size_t pivot = k;
30 |         for (std::size_t i = k + 1; i < n; ++i) {
31 |             if (std::abs(upper(i, k)) > std::abs(upper(pivot, k))) {
32 |                 pivot = i;
33 |             }
34 |         }
35 |         if (std::abs(upper(pivot, k)) <= threshold) {
36 |             throw std::domain_error("matrix is singular");
37 |         }
38 |         if (pivot != k) {
39 |             upper.swapRows(k, pivot);
40 |             for (std::size_t j = 0; j < k; ++j) {
41 |                 std::swap(lower(k, j), lower(pivot, j));
42 |             }
43 |             std::swap(result.permutation[k], result.permutation[pivot]);
44 |             result.sign = -result.sign;
45 |         }
46 |         for (std::size_t i = k + 1; i < n; ++i) {
47 |             const double factor = upper(i, k) / upper(k, k);

```

```

48         lower(i, k) = factor;
49         upper(i, k) = 0.0;
50         for (std::size_t j = k + 1; j < n; ++j) {
51             upper(i, j) -= factor * upper(k, j);
52         }
53     }
54 }
55 return result;
56 }
57
58 Vector solveLu(const LuDecomposition& lu, const Vector& rhs) {
59     const std::size_t n = lu.upper.rows();
60     if (rhs.size() != n) {
61         throw std::invalid_argument("right-hand side size does not match the matrix");
62     }
63     Vector solution(n);
64     for (std::size_t i = 0; i < n; ++i) {
65         double sum = rhs[lu.permutation[i]];
66         for (std::size_t j = 0; j < i; ++j) {
67             sum -= lu.lower(i, j) * solution[j];
68         }
69         solution[i] = sum;
70     }
71     for (std::size_t i = n; i-- > 0;) {
72         double sum = solution[i];
73         for (std::size_t j = i + 1; j < n; ++j) {
74             sum -= lu.upper(i, j) * solution[j];
75         }
76         solution[i] = sum / lu.upper(i, i);
77     }
78     return solution;
79 }
80
81 double determinant(const LuDecomposition& lu) {
82     double result = lu.sign;
83     for (std::size_t i = 0; i < lu.upper.rows(); ++i) {
84         result *= lu.upper(i, i);
85     }
86     return result;
87 }
88
89 Matrix inverse(const LuDecomposition& lu) {
90     const std::size_t n = lu.upper.rows();
91     Matrix result(n, n);
92     Vector unit(n);
93     for (std::size_t j = 0; j < n; ++j) {
94         unit.assign(n, 0.0);
95         unit[j] = 1.0;
96         const Vector column = solveLu(lu, unit);
97         for (std::size_t i = 0; i < n; ++i) {
98             result(i, j) = column[i];
99         }
100     }
101     return result;
102 }

```

2 Thomas algorithm

2.1 Task

Implement the Thomas algorithm (tridiagonal matrix algorithm) taking the non-zero elements of the matrix and the right-hand side as input. Use it to solve the system with a tridiagonal matrix.

$$\begin{cases} 16x_1 - 9x_2 = -27 \\ 8x_1 - 13x_2 - 5x_3 = -84 \\ -3x_2 - 21x_3 + 9x_4 = -225 \\ -9x_3 + 16x_4 - 5x_5 = -89 \\ x_4 - 9x_5 = 69 \end{cases}$$

2.2 Method

Row i of the system is $a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$ with $a_1 = c_n = 0$. The solution is sought as $x_i = P_i x_{i+1} + Q_i$. The forward sweep computes

$$P_i = \frac{-c_i}{b_i + a_i P_{i-1}}, \quad Q_i = \frac{d_i - a_i Q_{i-1}}{b_i + a_i P_{i-1}}, \quad P_0 = Q_0 = 0,$$

and the backward sweep gives $x_n = Q_n$ and $x_i = P_i x_{i+1} + Q_i$ for $i = n-1, \dots, 1$. Both sweeps take $O(n)$ operations and $O(n)$ memory.

The sufficient condition of correctness and stability is $a_i \neq 0$, $c_i \neq 0$ and diagonal dominance $|b_i| \geq |a_i| + |c_i|$ for every i with a strict inequality for at least one i . Then $|P_i| \leq 1$, the denominators do not vanish and rounding errors do not grow in the backward sweep. Without the requirement $a_i, c_i \neq 0$ dominance is not enough: the system can split into independent blocks, one of which is singular. The program checks this condition and reports it, but it also solves systems that do not satisfy it as long as no denominator is zero.

2.3 Results

The program reads n , the sub-diagonal $a_2, \dots, a_n$, the diagonal $b_1, \dots, b_n$, the super-diagonal $c_1, \dots, c_{n-1}$ and the right-hand side $d_1, \dots, d_n$.

Listing 5: tests/data/tridiagonal/01.in

```
5
8 -3 -9 1
16 -13 -21 16 -9
-9 -5 9 -5
-27 -84 -225 -89 69
```

Listing 6: tests/data/tridiagonal/01.out

```
x = (0.0000, 3.0000, 9.0000, -3.0000, -8.0000)
Stability condition: yes
```

The solution is $x = (0, 3, 9, -3, -8)^T$. The matrix of the variant is diagonally dominant, the second row turns the condition into an equality: $|-13| = |8| + |-5|$. The unit tests compare the Thomas algorithm with the LU decomposition on random systems up to 12×12 .

2.4 Source code

Listing 7: include/tridiagonal.hpp

```
1 | #pragma once
2 |
3 | #include "matrix.hpp"
4 |
5 | struct TridiagonalSystem {
6 |     Vector lower;
7 |     Vector diagonal;
8 |     Vector upper;
9 |     Vector rhs;
10 | };
11 |
12 | Vector solveTridiagonal(const TridiagonalSystem& system);
13 | bool meetsStabilityCondition(const TridiagonalSystem& system);
```

Listing 8: src/tridiagonal.cpp

```
1 | #include "tridiagonal.hpp"
2 |
3 | #include <algorithm>
4 | #include <cmath>
5 | #include <cstdint>
6 | #include <stdexcept>
7 |
8 | namespace {
9 |
10 | constexpr double SINGULARITY_TOLERANCE = 1e-12;
11 |
```

```

12 void validate(const TridiagonalSystem& system) {
13     const std::size_t n = system.diagonal.size();
14     if (n == 0 || system.rhs.size() != n || system.lower.size() + 1 != n ||
15         system.upper.size() + 1 != n) {
16         throw std::invalid_argument("tridiagonal system has inconsistent sizes");
17     }
18 }
19
20 double lowerAt(const TridiagonalSystem& system, std::size_t row) {
21     return row == 0 ? 0.0 : system.lower[row - 1];
22 }
23
24 double upperAt(const TridiagonalSystem& system, std::size_t row) {
25     return row + 1 == system.diagonal.size() ? 0.0 : system.upper[row];
26 }
27
28 }
29
30 Vector solveTridiagonal(const TridiagonalSystem& system) {
31     validate(system);
32     const std::size_t n = system.diagonal.size();
33     double scale = 0.0;
34     for (std::size_t i = 0; i < n; ++i) {
35         scale = std::max({scale, std::abs(lowerAt(system, i)), std::abs(system.diagonal[i]),
36                               std::abs(upperAt(system, i))});
37     }
38
39     Vector p(n);
40     Vector q(n);
41     for (std::size_t i = 0; i < n; ++i) {
42         const double a = lowerAt(system, i);
43         const double previousP = i == 0 ? 0.0 : p[i - 1];
44         const double previousQ = i == 0 ? 0.0 : q[i - 1];
45         const double denominator = system.diagonal[i] + a * previousP;
46         if (std::abs(denominator) <= SINGULARITY_TOLERANCE * scale) {
47             throw std::domain_error("zero denominator in the Thomas algorithm");
48         }
49         p[i] = -upperAt(system, i) / denominator;
50         q[i] = (system.rhs[i] - a * previousQ) / denominator;
51     }
52
53     Vector solution(n);
54     solution[n - 1] = q[n - 1];
55     for (std::size_t i = n - 1; i-- > 0;) {
56         solution[i] = p[i] * solution[i + 1] + q[i];
57     }
58     return solution;
59 }
60
61 bool meetsStabilityCondition(const TridiagonalSystem& system) {
62     validate(system);
63     const auto isZero = [](double value) {
64         return value == 0.0;
65     };
66     if (std::ranges::any_of(system.lower, isZero) || std::ranges::any_of(system.upper, isZero)) {
67         return false;
68     }
69     bool strict = false;
70     for (std::size_t i = 0; i < system.diagonal.size(); ++i) {
71         const double diagonal = std::abs(system.diagonal[i]);
72         const double offDiagonal = std::abs(lowerAt(system, i)) + std::abs(upperAt(system, i));
73         if (diagonal < offDiagonal) {

```

```
74 |         return false;
75 |     }
76 |     strict = strict || diagonal > offDiagonal;
77 | }
78 | return strict;
79 | }
```

3 Simple iteration and Seidel methods

3.1 Task

Implement the simple iteration method and the Seidel method taking the matrix, the right-hand side and the precision as input. Solve the system and analyze the number of iterations needed to reach the given precision.

$$\begin{cases} 10x_1 + 2x_3 + 4x_4 = 110 \\ 2x_1 + 16x_2 - 3x_3 + 8x_4 = 128 \\ x_1 + 5x_2 + 11x_3 - 4x_4 = 102 \\ 8x_1 + x_2 + 6x_3 - 17x_4 = 81 \end{cases}$$

3.2 Method

The system is rewritten in the equivalent form $x = \beta + \alpha x$, where $\beta_i = b_i/a_{ii}$, $\alpha_{ij} = -a_{ij}/a_{ii}$ for $i \neq j$ and $\alpha_{ii} = 0$. Both methods start from $x^{(0)} = \beta$.

The simple iteration method computes $x^{(k+1)} = \beta + \alpha x^{(k)}$. The Seidel method uses the components that are already updated in the current iteration:

$$x_i^{(k+1)} = \beta_i + \sum_{j < i} \alpha_{ij} x_j^{(k+1)} + \sum_{j > i} \alpha_{ij} x_j^{(k)}.$$

If $\|\alpha\| < 1$, both methods converge for any initial vector, and the iterations stop when the a posteriori estimate of the error does not exceed ε :

$$\varepsilon^{(k)} = \frac{\|\alpha\|}{1 - \|\alpha\|} \|x^{(k)} - x^{(k-1)}\| \leq \varepsilon, \quad \varepsilon^{(k)} = \frac{\|C\|}{1 - \|\alpha\|} \|x^{(k)} - x^{(k-1)}\| \leq \varepsilon,$$

for the simple iteration and the Seidel method respectively, where C is the upper triangular part of α ($c_{ij} = \alpha_{ij}$ for $j > i$). If $\|\alpha\| \geq 1$, the criterion is $\|x^{(k)} - x^{(k-1)}\| \leq \varepsilon$. All norms are $\|\cdot\|_\infty$. The a priori estimate of the number of simple iterations is

$$k + 1 \geq \frac{\lg \varepsilon - \lg \|\beta\| + \lg(1 - \|\alpha\|)}{\lg \|\alpha\|}.$$

A zero on the diagonal makes the transformation impossible. A non-finite iterate or 10^5 iterations without reaching the precision mean that the method does not converge. The program reports this for each method separately and fails only if neither method converges.

3.3 Results

The program reads n , the matrix A , the vector b and the precision ε .

Listing 9: tests/data/iterative/01.in

```
4
10 0 2 4
2 16 -3 8
1 5 11 -4
8 1 6 -17
110 128 102 81
0.0001
```

Listing 10: tests/data/iterative/01.out

```
||alpha|| = 0.9091
A priori estimate: 146 iterations
Simple iteration: 22 iterations
x = (9.0000, 7.0000, 6.0000, 2.0000)
Seidel: 12 iterations
x = (9.0000, 7.0000, 6.0000, 2.0000)
```

The exact solution is $x = (9, 7, 6, 2)^T$. The matrix is diagonally dominant and $\|\alpha\|_\infty = 10/11 \approx 0.9091$, so both methods converge.

3.4 Analysis

ε	A priori estimate	Simple iteration	Seidel
10^{-1}	74	12	7
10^{-2}	98	15	9
10^{-3}	122	18	10
10^{-4}	146	22	12
10^{-5}	171	26	14
10^{-6}	195	29	15
10^{-7}	219	33	17
10^{-8}	243	36	19
10^{-9}	267	40	20
10^{-10}	291	43	22

Table 1: Number of iterations depending on the precision

The number of iterations grows linearly in $\lg(1/\varepsilon)$: about 3.5 iterations per decimal digit for the simple iteration and about 1.7 for the Seidel method. The rate is determined by the spectral radius of the iteration matrix, $\rho(\alpha) \approx 0.518$, which gives $1/\lg(1/0.518) \approx 3.5$. For the Seidel method the spectral radius is $\rho \approx 0.257 \approx \rho(\alpha)^2$, so it needs about half as many iterations.

The a priori estimate is correct but about 7 times too pessimistic: it uses $\|\alpha\|_\infty \approx 0.909$, which is much larger than $\rho(\alpha)$. The estimate is useful to guarantee termination, while the a posteriori criterion stops the process in time.

The condition $\|\alpha\| < 1$ is only sufficient. For the matrix with ones on the diagonal and 0.5 elsewhere $\|\alpha\|_\infty = \rho(\alpha) = 1$: the error along $(1, 1, 1)^T$ changes its sign at every step and the simple iteration never converges, while the Seidel method converges, because the matrix is symmetric and positive definite.

Listing 11: tests/data/iterative/03.out

```
||alpha|| = 1.0000
A priori estimate: none
Simple iteration: no convergence after 100000 iterations
Seidel: 11 iterations
x = (1.0000, 1.0000, 1.0000)
```

3.5 Source code

Listing 12: include/iterative.hpp

```
1  #pragma once
2
3  #include <cstdlib>
4  #include <optional>
5
6  #include "matrix.hpp"
7
8  struct IterationForm {
9      Matrix alpha;
10     Vector beta;
11 };
12
13 struct IterationResult {
14     Vector solution;
15     std::size_t iterations = 0;
16     bool converged = false;
17 };
18
19 IterationForm toIterationForm(const Matrix& matrix, const Vector& rhs);
20 IterationResult simpleIteration(const Matrix& matrix, const Vector& rhs, double eps);
21 IterationResult seidel(const Matrix& matrix, const Vector& rhs, double eps);
22 std::optional<std::size_t> aprioriIterations(const Matrix& matrix, const Vector& rhs, double eps);
```

Listing 13: src/iterative.cpp

```

1  #include "iterative.hpp"
2
3  #include <algorithm>
4  #include <cmath>
5  #include <limits>
6  #include <stdexcept>
7  #include <utility>
8
9  namespace {
10
11  constexpr std::size_t MAX_ITERATIONS = 100000;
12
13  double upperNormInf(const Matrix& matrix) {
14      double result = 0.0;
15      for (std::size_t i = 0; i < matrix.rows(); ++i) {
16          double rowSum = 0.0;
17          for (std::size_t j = i + 1; j < matrix.cols(); ++j) {
18              rowSum += std::abs(matrix(i, j));
19          }
20          result = std::max(result, rowSum);
21      }
22      return result;
23  }
24
25  bool isFinite(const Vector& vector) {
26      for (const double value : vector) {
27          if (!std::isfinite(value)) {
28              return false;
29          }
30      }
31      return true;
32  }
33
34  template <typename Step>
35  IterationResult iterate(const IterationForm& form, double eps, double factor, Step step) {
36      IterationResult result{.solution = form.beta, .iterations = 0, .converged = false};
37      while (result.iterations < MAX_ITERATIONS) {
38          Vector next = step(result.solution);
39          ++result.iterations;
40          if (!isFinite(next)) {
41              return result;
42          }
43          const double error = factor * distanceInf(next, result.solution);
44          result.solution = std::move(next);
45          if (error <= eps) {
46              result.converged = true;
47              return result;
48          }
49      }
50      return result;
51  }
52
53  }
54
55  IterationForm toIterationForm(const Matrix& matrix, const Vector& rhs) {
56      if (!matrix.isSquare() || matrix.rows() == 0 || rhs.size() != matrix.rows()) {
57          throw std::invalid_argument("iterative methods need a square matrix and a matching rhs");
58      }
59      const std::size_t n = matrix.rows();

```

```

60     IterationForm form{.alpha = Matrix(n, n), .beta = Vector(n)};
61     for (std::size_t i = 0; i < n; ++i) {
62         const double diagonal = matrix(i, i);
63         if (diagonal == 0.0) {
64             throw std::domain_error("zero on the main diagonal");
65         }
66         form.beta[i] = rhs[i] / diagonal;
67         for (std::size_t j = 0; j < n; ++j) {
68             form.alpha(i, j) = i == j ? 0.0 : -matrix(i, j) / diagonal;
69         }
70     }
71     return form;
72 }
73
74 IterationResult simpleIteration(const Matrix& matrix, const Vector& rhs, double eps) {
75     const IterationForm form = toIterationForm(matrix, rhs);
76     const double alphaNorm = normInf(form.alpha);
77     const double factor = alphaNorm < 1.0 ? alphaNorm / (1.0 - alphaNorm) : 1.0;
78     return iterate(form, eps, factor, [&form](const Vector& current) {
79         Vector next = form.alpha * current;
80         for (std::size_t i = 0; i < next.size(); ++i) {
81             next[i] += form.beta[i];
82         }
83         return next;
84     });
85 }
86
87 IterationResult seidel(const Matrix& matrix, const Vector& rhs, double eps) {
88     const IterationForm form = toIterationForm(matrix, rhs);
89     const double alphaNorm = normInf(form.alpha);
90     const double factor = alphaNorm < 1.0 ? upperNormInf(form.alpha) / (1.0 - alphaNorm) : 1.0;
91     return iterate(form, eps, factor, [&form](const Vector& current) {
92         Vector next = current;
93         for (std::size_t i = 0; i < next.size(); ++i) {
94             double value = form.beta[i];
95             for (std::size_t j = 0; j < next.size(); ++j) {
96                 value += form.alpha(i, j) * next[j];
97             }
98             next[i] = value;
99         }
100         return next;
101     });
102 }
103
104 std::optional<std::size_t> aprioriIterations(const Matrix& matrix, const Vector& rhs, double eps) {
105     const IterationForm form = toIterationForm(matrix, rhs);
106     const double alphaNorm = normInf(form.alpha);
107     const double betaNorm = normInf(form.beta);
108     if (alphaNorm >= 1.0) {
109         return std::nullopt;
110     }
111     if (alphaNorm == 0.0 || betaNorm == 0.0) {
112         return 0;
113     }
114     const double bound =
115         (std::log(eps) - std::log(betaNorm) + std::log(1.0 - alphaNorm)) / std::log(alphaNorm);
116     if (!std::isfinite(bound) ||
117         bound >= static_cast<double>(std::numeric_limits<std::size_t>::max())) {
118         return std::nullopt;
119     }
120     return bound <= 1.0 ? 0 : static_cast<std::size_t>(std::ceil(bound - 1.0));
121 }

```

4 Jacobi rotation method

4.1 Task

Implement the rotation method taking the matrix and the precision as input. Find the eigenvalues and the eigenvectors of the symmetric matrix and analyze the dependence of the error on the number of iterations.

$$A = \begin{pmatrix} 3 & 2 & 6 \\ 2 & -3 & -7 \\ 6 & -7 & 3 \end{pmatrix}$$

4.2 Method

The method builds a sequence of similar matrices $A^{(k+1)} = U^{(k)T} A^{(k)} U^{(k)}$, where $U^{(k)}$ is the rotation in the plane (i, j) of the largest off-diagonal element $|a_{ij}^{(k)}|$: $u_{ii} = u_{jj} = \cos \varphi_k$, $u_{ij} = -\sin \varphi_k$, $u_{ji} = \sin \varphi_k$. The angle makes the element $a_{ij}^{(k+1)}$ zero:

$$\varphi_k = \frac{1}{2} \arctan \frac{2a_{ij}^{(k)}}{a_{ii}^{(k)} - a_{jj}^{(k)}}, \quad \varphi_k = \frac{\pi}{4} \operatorname{sign} a_{ij}^{(k)} \text{ if } a_{ii}^{(k)} = a_{jj}^{(k)}.$$

A rotation changes only rows and columns i and j , so one step costs $O(n)$ plus $O(n^2)$ for the search of the pivot. The error is measured by the norm of the off-diagonal part

$$t(A^{(k)}) = \left(\sum_{i < j} (a_{ij}^{(k)})^2 \right)^{1/2},$$

and the process stops when $t(A^{(k)}) \leq \varepsilon$. The diagonal of $A^{(k)}$ contains the eigenvalues and the columns of $U = U^{(0)} U^{(1)} \dots U^{(k-1)}$ are the orthonormal eigenvectors.

4.3 Results

The program reads n , the matrix A and the precision ε and prints the error after every rotation, the eigenvalues and the eigenvectors.

Listing 14: tests/data/rotation/01.in

```
3
3 2 6
2 -3 -7
6 -7 3
0.000001
```

Listing 15: tests/data/rotation/01.out

```
Rotations: 8
k  off(A)
0  9.4340e+00
1  6.3246e+00
2  3.9081e+00
3  1.4365e+00
4  8.2361e-01
5  4.8846e-02
6  3.3034e-03
7  2.0768e-05
8  3.4412e-09
lambda_1 = 2.4775, x_1 = (0.7410, 0.6150, -0.2695)
lambda_2 = -9.7068, x_2 = (-0.3862, 0.7187, 0.5783)
lambda_3 = 10.2293, x_3 = (0.5494, -0.3244, 0.7700)
```

The unit tests check $Ax_i = \lambda_i x_i$ and $U^T U = E$ on random symmetric matrices up to 7×7 .

4.4 Analysis

A rotation preserves the Frobenius norm and moves the weight of the element a_{ij} to the diagonal, so $t^2(A^{(k+1)}) = t^2(A^{(k)}) - (a_{ij}^{(k)})^2$ and the error decreases monotonically (the first step of the table removes $a_{23} = -7$: $9.434^2 - 6.325^2 = 49$). Since the largest of the $n(n-1)/2$ elements is chosen, $a_{ij}^2 \geq \frac{2}{n(n-1)} t^2$, and the convergence is at least linear:

$$t(A^{(k)}) \leq \left(1 - \frac{2}{n(n-1)}\right)^{k/2} t(A^{(0)}).$$

For $n = 3$ the guaranteed factor is $\sqrt{2/3} \approx 0.82$ per rotation, but the table shows a faster decrease. The first four rotations reduce the error by 1.5–2.7 times, and then the

convergence becomes quadratic: $4.9 \cdot 10^{-2} \rightarrow 3.3 \cdot 10^{-3} \rightarrow 2.1 \cdot 10^{-5} \rightarrow 3.4 \cdot 10^{-9}$, the number of correct digits roughly doubles with every rotation. That is why 8 rotations are enough for $\varepsilon = 10^{-6}$, and a higher precision costs only one or two extra rotations.

4.5 Source code

Listing 16: include/rotation.hpp

```

1  #pragma once
2
3  #include <cstddef>
4
5  #include "matrix.hpp"
6
7  struct EigenSystem {
8      Vector values;
9      Matrix vectors;
10     Vector errors;
11     std::size_t rotations = 0;
12 };
13
14 double offDiagonalNorm(const Matrix& matrix);
15 EigenSystem rotationMethod(const Matrix& matrix, double eps);

```

Listing 17: src/rotation.cpp

```

1  #include "rotation.hpp"
2
3  #include <cmath>
4  #include <numbers>
5  #include <stdexcept>
6
7  namespace {
8
9  constexpr std::size_t MAX_ROTATIONS = 100000;
10 constexpr double SYMMETRY_TOLERANCE = 1e-12;
11
12 void rotateColumns(Matrix& matrix, std::size_t i, std::size_t j, double cosine, double sine) {
13     for (std::size_t k = 0; k < matrix.rows(); ++k) {
14         const double left = matrix(k, i);
15         const double right = matrix(k, j);
16         matrix(k, i) = cosine * left + sine * right;
17         matrix(k, j) = -sine * left + cosine * right;
18     }
19 }
20
21 void rotateRows(Matrix& matrix, std::size_t i, std::size_t j, double cosine, double sine) {
22     for (std::size_t k = 0; k < matrix.cols(); ++k) {
23         const double top = matrix(i, k);
24         const double bottom = matrix(j, k);
25         matrix(i, k) = cosine * top + sine * bottom;
26         matrix(j, k) = -sine * top + cosine * bottom;
27     }
28 }
29
30 }

```

```

31
32 double offDiagonalNorm(const Matrix& matrix) {
33     double sum = 0.0;
34     for (std::size_t i = 0; i < matrix.rows(); ++i) {
35         for (std::size_t j = i + 1; j < matrix.cols(); ++j) {
36             sum += matrix(i, j) * matrix(i, j);
37         }
38     }
39     return std::sqrt(sum);
40 }
41
42 EigenSystem rotationMethod(const Matrix& matrix, double eps) {
43     if (matrix.rows() == 0 || !isSymmetric(matrix, SYMMETRY_TOLERANCE * normInf(matrix))) {
44         throw std::invalid_argument("rotation method needs a symmetric matrix");
45     }
46     const std::size_t n = matrix.rows();
47     Matrix a = matrix;
48     EigenSystem result{.values = Vector(n),
49                       .vectors = Matrix::identity(n),
50                       .errors = {offDiagonalNorm(a)},
51                       .rotations = 0};
52
53     while (result.errors.back() > eps) {
54         if (result.rotations == MAX_ROTATIONS) {
55             throw std::runtime_error("rotation method did not converge");
56         }
57         std::size_t pivotRow = 0;
58         std::size_t pivotCol = 1;
59         for (std::size_t i = 0; i < n; ++i) {
60             for (std::size_t j = i + 1; j < n; ++j) {
61                 if (std::abs(a(i, j)) > std::abs(a(pivotRow, pivotCol))) {
62                     pivotRow = i;
63                     pivotCol = j;
64                 }
65             }
66         }
67         const double pivot = a(pivotRow, pivotCol);
68         const double difference = a(pivotRow, pivotRow) - a(pivotCol, pivotCol);
69         const double angle = difference == 0.0 ? std::copysign(std::numbers::pi / 4.0, pivot)
70             : 0.5 * std::atan(2.0 * pivot / difference);
71         const double cosine = std::cos(angle);
72         const double sine = std::sin(angle);
73         rotateColumns(a, pivotRow, pivotCol, cosine, sine);
74         rotateRows(a, pivotRow, pivotCol, cosine, sine);
75         a(pivotRow, pivotCol) = 0.0;
76         a(pivotCol, pivotRow) = 0.0;
77         rotateColumns(result.vectors, pivotRow, pivotCol, cosine, sine);
78         ++result.rotations;
79         result.errors.push_back(offDiagonalNorm(a));
80     }
81
82     for (std::size_t i = 0; i < n; ++i) {
83         result.values[i] = a(i, i);
84     }
85     return result;
86 }

```

5 QR algorithm

5.1 Task

Implement the QR decomposition of a matrix. Based on it, implement the QR algorithm for the full eigenvalue problem of an arbitrary matrix taking the matrix and the precision as input, and find the eigenvalues of the matrix.

$$A = \begin{pmatrix} 6 & -3 & 7 \\ 9 & 1 & -6 \\ 3 & -5 & 5 \end{pmatrix}$$

5.2 Method

The QR decomposition $A = QR$ with an orthogonal Q and an upper triangular R is built with Householder reflections. At step $k = 1, \dots, n - 1$ the vector v is formed from the part of column k on and below the diagonal:

$$v = a_k + \text{sign}(a_{kk})\|a_k\|_2 e_k, \quad H_k = E - \frac{2vv^T}{v^T v}.$$

The reflection H_k zeroes the elements below the diagonal in column k , and the choice of the sign avoids cancellation. Then $R = H_{n-1} \cdots H_1 A$ and $Q = H_1 \cdots H_{n-1}$. The reflections are applied as rank-one updates without forming H_k .

The QR algorithm builds the sequence $A^{(0)} = A$, $A^{(k)} = Q^{(k)} R^{(k)}$, $A^{(k+1)} = R^{(k)} Q^{(k)} = Q^{(k)T} A^{(k)} Q^{(k)}$ of similar matrices. It converges to a block upper triangular form: 1×1 blocks hold real eigenvalues and 2×2 blocks hold pairs of complex conjugate eigenvalues. The diagonal is scanned column by column:

- if $\left(\sum_{i>j} a_{ij}^2\right)^{1/2} \leq \varepsilon$, then a_{jj} is a real eigenvalue;
- otherwise columns $j, j + 1$ form a block, whose eigenvalues are the roots of $(a_{jj} - \lambda)(a_{j+1,j+1} - \lambda) = a_{j,j+1}a_{j+1,j}$. The block is accepted when the elements below it are smaller than ε and its eigenvalues differ from those of the previous iteration by at most ε .

The iterations stop when every diagonal position is covered by an accepted block. For complex eigenvalues the sub-diagonal element of the block does not vanish, so the stability of the block eigenvalues is the only possible criterion.

5.3 Results

The program reads n , the matrix A and the precision ε and prints the QR decomposition of A , the number of iterations and the eigenvalues.

Listing 18: tests/data/qr/01.in

```
3
6 -3 7
9 1 -6
3 -5 5
0.000001
```

Listing 19: tests/data/qr/01.out

```
Q =
-0.5345  0.3367 -0.7752
-0.8018 -0.4921  0.3392
-0.2673  0.8028  0.5330
R =
-11.2250  2.1381 -0.2673
  0.0000 -5.5162  9.3232
  0.0000  0.0000 -4.7966
Iterations: 20
lambda_1 = 7.9140 + 3.8675i
lambda_2 = 7.9140 - 3.8675i
lambda_3 = -3.8279
```

The matrix has one real eigenvalue and a pair of complex conjugate eigenvalues. The unit tests check $QR = A$ and $Q^T Q = E$, compare the eigenvalues of symmetric matrices with the rotation method and check that the sum and the product of the eigenvalues are equal to the trace and the determinant of random matrices up to 6×6 .

5.4 Analysis

ε	10^{-1}	10^{-2}	10^{-3}	10^{-4}	10^{-8}
Iterations	7	9	12	15	26

Table 2: Number of QR iterations depending on the precision

The elements below the 2×2 block decrease as $(|\lambda_3|/|\lambda_{1,2}|)^k = (3.83/8.81)^k \approx 0.435^k$, which gives $1/\lg(1/0.435) \approx 2.8$ iterations per decimal digit, in agreement with the table.

5.5 Source code

Listing 20: include/qr.hpp

```
1  #pragma once
2
3  #include <complex>
4  #include <cstdint>
5  #include <vector>
6
7  #include "matrix.hpp"
8
9  struct QrDecomposition {
10     Matrix q;
11     Matrix r;
12 };
13
14 struct QrEigenvalues {
15     std::vector<std::complex<double>> values;
16     std::size_t iterations = 0;
17 };
18
19 QrDecomposition decomposeQr(const Matrix& matrix);
20 QrEigenvalues qrAlgorithm(const Matrix& matrix, double eps);
```

Listing 21: src/qr.cpp

```
1  #include "qr.hpp"
2
3  #include <cmath>
4  #include <optional>
5  #include <stdexcept>
6  #include <utility>
7
8  namespace {
9
10     constexpr std::size_t MAX_ITERATIONS = 100000;
11
12     using Complex = std::complex<double>;
13     using ComplexPair = std::pair<Complex, Complex>;
14
15     double columnTail(const Matrix& matrix, std::size_t col, std::size_t fromRow) {
16         double sum = 0.0;
17         for (std::size_t i = fromRow; i < matrix.rows(); ++i) {
18             sum += matrix(i, col) * matrix(i, col);
19         }
20         return std::sqrt(sum);
21     }
22
23     ComplexPair blockEigenvalues(const Matrix& matrix, std::size_t j) {
24         const double a = matrix(j, j);
25         const double b = matrix(j, j + 1);
26         const double c = matrix(j + 1, j);
27         const double d = matrix(j + 1, j + 1);
28         const double halfTrace = (a + d) / 2.0;
29         const Complex root = std::sqrt(Complex((a - d) * (a - d) / 4.0 + b * c, 0.0));
30         return {halfTrace + root, halfTrace - root};
31     }
32 }
```

```

33 bool isStable(const ComplexPair& current, const std::optional<ComplexPair>& last, double eps) {
34     if (!last) {
35         return false;
36     }
37     return std::abs(current.first - last->first) <= eps &&
38         std::abs(current.second - last->second) <= eps;
39 }
40
41 std::optional<std::vector<Complex>>> extractEigenvalues(
42     const Matrix& matrix, double eps, std::vector<std::optional<ComplexPair>>& previous) {
43     const std::size_t n = matrix.rows();
44     std::vector<Complex> values;
45     values.reserve(n);
46     bool converged = true;
47     std::size_t j = 0;
48     while (j < n) {
49         if (j + 1 == n || columnTail(matrix, j, j + 1) <= eps) {
50             values.emplace_back(matrix(j, j), 0.0);
51             ++j;
52             continue;
53         }
54         const ComplexPair current = blockEigenvalues(matrix, j);
55         const bool isolated =
56             columnTail(matrix, j, j + 2) <= eps && columnTail(matrix, j + 1, j + 2) <= eps;
57         const bool stable = isStable(current, previous[j], eps);
58         previous[j] = current;
59         converged = converged && isolated && stable;
60         values.push_back(current.first);
61         values.push_back(current.second);
62         j += 2;
63     }
64     if (!converged) {
65         return std::nullopt;
66     }
67     return values;
68 }
69
70 }
71
72 QrDecomposition decomposeQr(const Matrix& matrix) {
73     if (!matrix.isSquare() || matrix.rows() == 0) {
74         throw std::invalid_argument("QR decomposition needs a non-empty square matrix");
75     }
76     const std::size_t n = matrix.rows();
77     QrDecomposition result{.q = Matrix::identity(n), .r = matrix};
78     Vector v(n);
79     for (std::size_t k = 0; k + 1 < n; ++k) {
80         const double norm = columnTail(result.r, k, k);
81         if (norm == 0.0) {
82             continue;
83         }
84         v.assign(n, 0.0);
85         for (std::size_t i = k; i < n; ++i) {
86             v[i] = result.r(i, k);
87         }
88         v[k] += std::copysign(norm, v[k]);
89         double squaredNorm = 0.0;
90         for (std::size_t i = k; i < n; ++i) {
91             squaredNorm += v[i] * v[i];
92         }
93         for (std::size_t j = 0; j < n; ++j) {
94             double dot = 0.0;

```

```

95         for (std::size_t i = k; i < n; ++i) {
96             dot += v[i] * result.r(i, j);
97         }
98         const double factor = 2.0 * dot / squaredNorm;
99         for (std::size_t i = k; i < n; ++i) {
100             result.r(i, j) -= factor * v[i];
101         }
102     }
103     for (std::size_t i = 0; i < n; ++i) {
104         double dot = 0.0;
105         for (std::size_t j = k; j < n; ++j) {
106             dot += result.q(i, j) * v[j];
107         }
108         const double factor = 2.0 * dot / squaredNorm;
109         for (std::size_t j = k; j < n; ++j) {
110             result.q(i, j) -= factor * v[j];
111         }
112     }
113     for (std::size_t i = k + 1; i < n; ++i) {
114         result.r(i, k) = 0.0;
115     }
116 }
117 return result;
118 }
119
120 QrEigenvalues qrAlgorithm(const Matrix& matrix, double eps) {
121     if (!matrix.isSquare() || matrix.rows() == 0) {
122         throw std::invalid_argument("QR algorithm needs a non-empty square matrix");
123     }
124     Matrix a = matrix;
125     std::vector<std::optional<ComplexPair>> previous(matrix.rows());
126     for (std::size_t iteration = 0; iteration <= MAX_ITERATIONS; ++iteration) {
127         if (auto values = extractEigenvalues(a, eps, previous)) {
128             return QrEigenvalues{.values = std::move(*values), .iterations = iteration};
129         }
130         const QrDecomposition qr = decomposeQr(a);
131         a = qr.r * qr.q;
132     }
133     throw std::runtime_error("QR algorithm did not converge");
134 }

```

6 Common code

All five programs share the library `lab1_core`. The class `Matrix` stores a dense matrix in a single vector in row-major order and provides the products and the norms. The module `io` reads the input with strict validation (the size from 1 to 1000, every number in $[-10^{100}, 10^{100}]$ so that norms and products cannot overflow, and the absence of trailing data) and formats numbers with four digits after the decimal point. The module `cli` contains one function per program; it reads the whole input, computes the result and only then prints it, so an error never leaves a partial output. Errors are written to the standard error stream as `error: <message>` with the exit code 1. Every executable in `app/` consists of a single call:

Listing 22: `app/lu.cpp`

```
1 | #include <iostream>
2 |
3 | #include "cli.hpp"
4 |
5 | int main() {
6 |     return runTask(runLu, std::cin, std::cout, std::cerr);
7 | }
```

Listing 23: `include/matrix.hpp`

```
1 | #pragma once
2 |
3 | #include <cstddef>
4 | #include <initializer_list>
5 | #include <vector>
6 |
7 | using Vector = std::vector<double>;
8 |
9 | class Matrix {
10 | public:
11 |     Matrix() = default;
12 |     Matrix(std::size_t rows, std::size_t cols);
13 |     Matrix(std::initializer_list<std::initializer_list<double>>> rows);
14 |
15 |     static Matrix identity(std::size_t size);
16 |
17 |     std::size_t rows() const { return rows_; }
18 |     std::size_t cols() const { return cols_; }
19 |     bool isSquare() const { return rows_ == cols_; }
20 |
21 |     double& operator()(std::size_t row, std::size_t col) { return data_[row * cols_ + col]; }
22 |     double operator()(std::size_t row, std::size_t col) const { return data_[row * cols_ + col]; }
23 |
24 |     Vector column(std::size_t col) const;
25 |     Matrix transposed() const;
26 |     void swapRows(std::size_t first, std::size_t second);
27 |
28 | private:
29 |     std::size_t rows_ = 0;
30 |     std::size_t cols_ = 0;
```

```

31     Vector data_;
32 };
33
34 Matrix operator*(const Matrix& lhs, const Matrix& rhs);
35 Vector operator*(const Matrix& matrix, const Vector& vector);
36
37 double normInf(const Vector& vector);
38 double normInf(const Matrix& matrix);
39 double distanceInf(const Vector& lhs, const Vector& rhs);
40 bool isSymmetric(const Matrix& matrix, double tolerance = 0.0);

```

Listing 24: src/matrix.cpp

```

1  #include "matrix.hpp"
2
3  #include <algorithm>
4  #include <cmath>
5  #include <stdexcept>
6  #include <utility>
7
8  Matrix::Matrix(std::size_t rows, std::size_t cols) : rows_(rows), cols_(cols), data_(rows * cols) {
9  }
10
11 Matrix::Matrix(std::initializer_list<std::initializer_list<double>> rows)
12 : rows_(rows.size()), cols_(rows.size() == 0 ? 0 : rows.begin()->size()) {
13     data_.reserve(rows_ * cols_);
14     for (const auto& row : rows) {
15         if (row.size() != cols_) {
16             throw std::invalid_argument("matrix rows have different lengths");
17         }
18         data_.insert(data_.end(), row.begin(), row.end());
19     }
20 }
21
22 Matrix Matrix::identity(std::size_t size) {
23     Matrix result(size, size);
24     for (std::size_t i = 0; i < size; ++i) {
25         result(i, i) = 1.0;
26     }
27     return result;
28 }
29
30 Vector Matrix::column(std::size_t col) const {
31     Vector result(rows_);
32     for (std::size_t i = 0; i < rows_; ++i) {
33         result[i] = (*this)(i, col);
34     }
35     return result;
36 }
37
38 Matrix Matrix::transposed() const {
39     Matrix result(cols_, rows_);
40     for (std::size_t i = 0; i < rows_; ++i) {
41         for (std::size_t j = 0; j < cols_; ++j) {
42             result(j, i) = (*this)(i, j);
43         }
44     }
45     return result;
46 }
47
48 void Matrix::swapRows(std::size_t first, std::size_t second) {

```

```

49     if (first == second) {
50         return;
51     }
52     for (std::size_t j = 0; j < cols_; ++j) {
53         std::swap((*this)(first, j), (*this)(second, j));
54     }
55 }
56
57 Matrix operator*(const Matrix& lhs, const Matrix& rhs) {
58     if (lhs.cols() != rhs.rows()) {
59         throw std::invalid_argument("matrix sizes do not match");
60     }
61     Matrix result(lhs.rows(), rhs.cols());
62     for (std::size_t i = 0; i < lhs.rows(); ++i) {
63         for (std::size_t k = 0; k < lhs.cols(); ++k) {
64             for (std::size_t j = 0; j < rhs.cols(); ++j) {
65                 result(i, j) += lhs(i, k) * rhs(k, j);
66             }
67         }
68     }
69     return result;
70 }
71
72 Vector operator*(const Matrix& matrix, const Vector& vector) {
73     if (matrix.cols() != vector.size()) {
74         throw std::invalid_argument("matrix and vector sizes do not match");
75     }
76     Vector result(matrix.rows());
77     for (std::size_t i = 0; i < matrix.rows(); ++i) {
78         for (std::size_t j = 0; j < matrix.cols(); ++j) {
79             result[i] += matrix(i, j) * vector[j];
80         }
81     }
82     return result;
83 }
84
85 double normInf(const Vector& vector) {
86     double result = 0.0;
87     for (const double value : vector) {
88         result = std::max(result, std::abs(value));
89     }
90     return result;
91 }
92
93 double normInf(const Matrix& matrix) {
94     double result = 0.0;
95     for (std::size_t i = 0; i < matrix.rows(); ++i) {
96         double rowSum = 0.0;
97         for (std::size_t j = 0; j < matrix.cols(); ++j) {
98             rowSum += std::abs(matrix(i, j));
99         }
100         result = std::max(result, rowSum);
101     }
102     return result;
103 }
104
105 double distanceInf(const Vector& lhs, const Vector& rhs) {
106     if (lhs.size() != rhs.size()) {
107         throw std::invalid_argument("vector sizes do not match");
108     }
109     double result = 0.0;
110     for (std::size_t i = 0; i < lhs.size(); ++i) {

```

```

111         result = std::max(result, std::abs(lhs[i] - rhs[i]));
112     }
113     return result;
114 }
115
116 bool isSymmetric(const Matrix& matrix, double tolerance) {
117     if (!matrix.isSquare()) {
118         return false;
119     }
120     for (std::size_t i = 0; i < matrix.rows(); ++i) {
121         for (std::size_t j = i + 1; j < matrix.cols(); ++j) {
122             if (std::abs(matrix(i, j) - matrix(j, i)) > tolerance) {
123                 return false;
124             }
125         }
126     }
127     return true;
128 }

```

Listing 25: include/io.hpp

```

1  #pragma once
2
3  #include <complex>
4  #include <cstdint>
5  #include <iosfwd>
6  #include <string>
7
8  #include "matrix.hpp"
9
10 std::size_t readSize(std::istream& input);
11 double readNumber(std::istream& input);
12 double readTolerance(std::istream& input);
13 Vector readVector(std::istream& input, std::size_t size);
14 Matrix readMatrix(std::istream& input, std::size_t size);
15 void expectEnd(std::istream& input);
16
17 std::string formatNumber(double value);
18 std::string formatScientific(double value);
19 std::string formatComplex(std::complex<double> value);
20 std::string formatVector(const Vector& vector);
21 void writeMatrix(std::ostream& output, const Matrix& matrix);

```

Listing 26: src/io.cpp

```

1  #include "io.hpp"
2
3  #include <algorithm>
4  #include <charconv>
5  #include <cmath>
6  #include <iomanip>
7  #include <istream>
8  #include <ostream>
9  #include <sstream>
10 #include <stdexcept>
11 #include <system_error>
12 #include <vector>
13
14 namespace {

```

```

15
16 constexpr int PRECISION = 4;
17 constexpr double ZERO_THRESHOLD = 5e-5;
18 constexpr std::size_t MAX_SIZE = 1000;
19 constexpr double MAX_ABS_VALUE = 1e100;
20
21 std::string readToken(std::istream& input) {
22     std::string token;
23     if (!(input >> token)) {
24         throw std::invalid_argument("unexpected end of input");
25     }
26     return token;
27 }
28
29 template <typename Number>
30 Number parseToken(const std::string& token, const char* expected) {
31     Number value{};
32     const bool hasPlus = token.size() > 1 && token[0] == '+' && token[1] != '-';
33     const char* begin = token.data() + (hasPlus ? 1 : 0);
34     const char* end = token.data() + token.size();
35     const auto [ptr, error] = std::from_chars(begin, end, value);
36     if (error != std::errc() || ptr != end) {
37         throw std::invalid_argument(std::string("expected ") + expected + ", got '" + token + "'");
38     }
39     return value;
40 }
41
42 }
43
44 std::size_t readSize(std::istream& input) {
45     const auto size = parseToken<std::size_t>(readToken(input), "a matrix size");
46     if (size == 0 || size > MAX_SIZE) {
47         throw std::invalid_argument("matrix size must be between 1 and " +
48                                     std::to_string(MAX_SIZE));
49     }
50     return size;
51 }
52
53 double readNumber(std::istream& input) {
54     const auto value = parseToken<double>(readToken(input), "a number");
55     if (!std::isfinite(value) || std::abs(value) > MAX_ABS_VALUE) {
56         throw std::invalid_argument("numbers must lie in [-1e100, 1e100]");
57     }
58     return value;
59 }
60
61 double readTolerance(std::istream& input) {
62     const double value = readNumber(input);
63     if (value <= 0.0) {
64         throw std::invalid_argument("precision must be positive");
65     }
66     return value;
67 }
68
69 Vector readVector(std::istream& input, std::size_t size) {
70     Vector result(size);
71     for (auto& value : result) {
72         value = readNumber(input);
73     }
74     return result;
75 }
76

```

```

77 Matrix readMatrix(std::istream& input, std::size_t size) {
78     Matrix result(size, size);
79     for (std::size_t i = 0; i < size; ++i) {
80         for (std::size_t j = 0; j < size; ++j) {
81             result(i, j) = readNumber(input);
82         }
83     }
84     return result;
85 }
86
87 void expectEnd(std::istream& input) {
88     std::string token;
89     if (input >> token) {
90         throw std::invalid_argument("unexpected trailing input '" + token + "'");
91     }
92 }
93
94 std::string formatNumber(double value) {
95     std::ostringstream stream;
96     stream << std::fixed << std::setprecision(PRECISION)
97         << (std::abs(value) < ZERO_THRESHOLD ? 0.0 : value);
98     return stream.str();
99 }
100
101 std::string formatScientific(double value) {
102     std::ostringstream stream;
103     stream << std::scientific << std::setprecision(PRECISION) << value;
104     return stream.str();
105 }
106
107 std::string formatComplex(std::complex<double> value) {
108     if (std::abs(value.imag()) < ZERO_THRESHOLD) {
109         return formatNumber(value.real());
110     }
111     return formatNumber(value.real()) + (value.imag() < 0.0 ? " - " : " + ") +
112         formatNumber(std::abs(value.imag())) + "i";
113 }
114
115 std::string formatVector(const Vector& vector) {
116     std::string result = "(";
117     for (std::size_t i = 0; i < vector.size(); ++i) {
118         if (i > 0) {
119             result += ", ";
120         }
121         result += formatNumber(vector[i]);
122     }
123     return result + ")";
124 }
125
126 void writeMatrix(std::ostream& output, const Matrix& matrix) {
127     std::vector<std::string> cells;
128     cells.reserve(matrix.rows() * matrix.cols());
129     std::size_t width = 0;
130     for (std::size_t i = 0; i < matrix.rows(); ++i) {
131         for (std::size_t j = 0; j < matrix.cols(); ++j) {
132             cells.push_back(formatNumber(matrix(i, j)));
133             width = std::max(width, cells.back().size());
134         }
135     }
136     const auto cellWidth = static_cast<int>(width);
137     for (std::size_t i = 0; i < matrix.rows(); ++i) {
138         for (std::size_t j = 0; j < matrix.cols(); ++j) {

```

```

139         output << " " << std::setw(cellWidth) << cells[i * matrix.cols() + j];
140     }
141     output << '\n';
142 }
143 }

```

Listing 27: include/cli.hpp

```

1  #pragma once
2
3  #include <iosfwd>
4
5  using Task = void (*)(std::istream& input, std::ostream& output);
6
7  int runTask(Task task, std::istream& input, std::ostream& output, std::ostream& error);
8
9  void runLu(std::istream& input, std::ostream& output);
10 void runTridiagonal(std::istream& input, std::ostream& output);
11 void runIterative(std::istream& input, std::ostream& output);
12 void runRotation(std::istream& input, std::ostream& output);
13 void runQr(std::istream& input, std::ostream& output);

```

Listing 28: src/cli.cpp

```

1  #include "cli.hpp"
2
3  #include <cstdint>
4  #include <exception>
5  #include <iomanip>
6  #include <istream>
7  #include <ostream>
8  #include <stdexcept>
9  #include <string>
10 #include <vector>
11
12 #include "io.hpp"
13 #include "iterative.hpp"
14 #include "lu.hpp"
15 #include "qr.hpp"
16 #include "rotation.hpp"
17 #include "tridiagonal.hpp"
18
19 namespace {
20
21 std::string formatIndices(const std::vector<std::size_t>& indices) {
22     std::string result = "(";
23     for (std::size_t i = 0; i < indices.size(); ++i) {
24         if (i > 0) {
25             result += ", ";
26         }
27         result += std::to_string(indices[i] + 1);
28     }
29     return result + ")";
30 }
31
32 void writeIterationResult(std::ostream& output, const char* name, const IterationResult& result) {
33     if (!result.converged) {
34         output << name << ": no convergence after " << result.iterations << " iterations\n";
35         return;
36     }
37 }

```

```

36     }
37     output << name << ": " << result.iterations << " iterations\n"
38         << "x = " << formatVector(result.solution) << '\n';
39 }
40
41 }
42
43 int runTask(Task task, std::istream& input, std::ostream& output, std::ostream& error) {
44     try {
45         task(input, output);
46         return 0;
47     } catch (const std::exception& exception) {
48         error << "error: " << exception.what() << '\n';
49         return 1;
50     }
51 }
52
53 void runLu(std::istream& input, std::ostream& output) {
54     const std::size_t n = readSize(input);
55     const Matrix matrix = readMatrix(input, n);
56     const Vector rhs = readVector(input, n);
57     expectEnd(input);
58
59     const LuDecomposition lu = decomposeLu(matrix);
60     const Vector solution = solveLu(lu, rhs);
61     const Matrix inverted = inverse(lu);
62
63     output << "P = " << formatIndices(lu.permutation) << '\n';
64     output << "L =\n";
65     writeMatrix(output, lu.lower);
66     output << "U =\n";
67     writeMatrix(output, lu.upper);
68     output << "x = " << formatVector(solution) << '\n';
69     output << "det(A) = " << formatNumber(determinant(lu)) << '\n';
70     output << "A^-1 =\n";
71     writeMatrix(output, inverted);
72 }
73
74 void runTridiagonal(std::istream& input, std::ostream& output) {
75     const std::size_t n = readSize(input);
76     TridiagonalSystem system;
77     system.lower = readVector(input, n - 1);
78     system.diagonal = readVector(input, n);
79     system.upper = readVector(input, n - 1);
80     system.rhs = readVector(input, n);
81     expectEnd(input);
82
83     const Vector solution = solveTridiagonal(system);
84
85     output << "x = " << formatVector(solution) << '\n';
86     output << "Stability condition: " << (meetsStabilityCondition(system) ? "yes" : "no") << '\n';
87 }
88
89 void runIterative(std::istream& input, std::ostream& output) {
90     const std::size_t n = readSize(input);
91     const Matrix matrix = readMatrix(input, n);
92     const Vector rhs = readVector(input, n);
93     const double eps = readTolerance(input);
94     expectEnd(input);
95
96     const IterationForm form = toIterationForm(matrix, rhs);
97     const IterationResult jacobi = simpleIteration(matrix, rhs, eps);

```

```

98     const IterationResult gaussSeidel = seidel(matrix, rhs, eps);
99     const auto estimate = aprioriIterations(matrix, rhs, eps);
100     if (!jacobi.converged && !gaussSeidel.converged) {
101         throw std::runtime_error("neither method converged");
102     }
103
104     output << "||alpha|| = " << formatNumber(normInf(form.alpha)) << '\n';
105     output << "A priori estimate: "
106         << (estimate ? std::to_string(*estimate) + " iterations" : std::string("none")) << '\n';
107     writeIterationResult(output, "Simple iteration", jacobi);
108     writeIterationResult(output, "Seidel", gaussSeidel);
109 }
110
111 void runRotation(std::istream& input, std::ostream& output) {
112     const std::size_t n = readSize(input);
113     const Matrix matrix = readMatrix(input, n);
114     const double eps = readTolerance(input);
115     expectEnd(input);
116
117     const EigenSystem eigen = rotationMethod(matrix, eps);
118
119     output << "Rotations: " << eigen.rotations << '\n';
120     output << "k off(A)\n";
121     for (std::size_t k = 0; k < eigen.errors.size(); ++k) {
122         output << std::setw(3) << k << " " << formatScientific(eigen.errors[k]) << '\n';
123     }
124     for (std::size_t i = 0; i < n; ++i) {
125         output << "lambda_" << i + 1 << " = " << formatNumber(eigen.values[i]) << ", x_" << i + 1
126             << " = " << formatVector(eigen.vectors.column(i)) << '\n';
127     }
128 }
129
130 void runQr(std::istream& input, std::ostream& output) {
131     const std::size_t n = readSize(input);
132     const Matrix matrix = readMatrix(input, n);
133     const double eps = readTolerance(input);
134     expectEnd(input);
135
136     const QrDecomposition qr = decomposeQr(matrix);
137     const QrEigenvalues eigen = qrAlgorithm(matrix, eps);
138
139     output << "Q =\n";
140     writeMatrix(output, qr.q);
141     output << "R =\n";
142     writeMatrix(output, qr.r);
143     output << "Iterations: " << eigen.iterations << '\n';
144     for (std::size_t i = 0; i < n; ++i) {
145         output << "lambda_" << i + 1 << " = " << formatComplex(eigen.values[i]) << '\n';
146     }
147 }

```

7 Conclusions

In this laboratory work for the course “Numerical Methods” I implemented the basic methods of numerical linear algebra and applied them to variant 28.

Direct methods give the solution in a fixed number of operations. The LU decomposition with partial pivoting solves a general system in $O(n^3)$, and the same factorization gives the determinant and the inverse matrix. The Thomas algorithm uses the structure of a tridiagonal matrix and solves the system in $O(n)$; diagonal dominance with non-zero off-diagonal elements guarantees its stability.

Iterative methods trade exactness for simplicity of every step. For the diagonally dominant system of the variant the Seidel method needed about half as many iterations as the simple iteration method, in agreement with the ratio of the spectral radii of their iteration matrices, while the a priori estimate turned out to be about 7 times too pessimistic.

The rotation method finds the complete eigensystem of a symmetric matrix: the error decreases monotonically and at the end converges quadratically. The QR algorithm with Householder reflections works for arbitrary matrices and finds complex eigenvalues as 2×2 blocks; its rate depends on the ratio of the moduli of the eigenvalues.

All results were verified by unit tests on random matrices (residuals, orthogonality, trace and determinant) and by end-to-end tests of the programs.

References

- [1] Golub G. H., Van Loan C. F. *Matrix Computations, 4th edition.* — Johns Hopkins University Press, 2013.
- [2] Demmel J. W. *Applied Numerical Linear Algebra.* — SIAM, 1997.
- [3] Burden R. L., Faires J. D. *Numerical Analysis, 9th edition.* — Brooks/Cole, 2011.